

DATA ANALYSIS INTERNSHIP

DAY 6 REPORT

NumPy Functions and Array Operations using Google Colab

1. Introduction to NumPy

NumPy is a Python library used for numerical computations, matrix operations, array manipulation, and mathematical calculations. It is widely used in Data Analytics, Machine Learning, and Artificial Intelligence applications.

```
import numpy as np
```

2. Array Creation Functions

`array()`

Creates a NumPy array.

```
np.array([1,2,3])
```

Output

```
array([1, 2, 3])
```

Explanation

Creates a one-dimensional array containing values 1, 2, and 3.

`zeros()`

Creates an array filled with zeros.

```
np.zeros((2,2))
```

Output

```
array([[0., 0.],  
       [0., 0.]])
```

Explanation

Creates a 2×2 matrix with all elements as 0.

`ones()`

Creates an array filled with ones.

```
np.ones((2,2))
```

Output

```
array([[1., 1.],  
       [1., 1.]])
```

Explanation

Creates a 2×2 matrix with all elements as 1.

```
empty()
```

Creates an empty array.

```
np.empty((2,2))
```

Output

```
array([[0., 0.],  
       [0., 0.]])
```

Explanation

Creates an empty array with random memory values. Output may vary each time.

```
full()
```

Creates an array filled with same values.

```
np.full((2,2),7)
```

Output

```
array([[7, 7],  
       [7, 7]])
```

Explanation

Creates a 2×2 matrix where every element is 7.

```
eye()
```

Creates an identity matrix.

```
np.eye(3)
```

Output

```
array([[1., 0., 0.],  
       [0., 1., 0.],  
       [0., 0., 1.]])
```

Explanation

Creates a 3×3 identity matrix where diagonal values are 1 and remaining values are 0.

arange()

Creates values within a range.

```
np.arange(1,10,2)
```

Output

```
array([1, 3, 5, 7, 9])
```

Explanation

Generates numbers from 1 to 9 with step size 2.

linspace()

Creates evenly spaced values.

```
np.linspace(0,10,5)
```

Output

```
array([ 0. ,  2.5,  5. ,  7.5, 10. ])
```

Explanation

Divides values equally between 0 and 10 into 5 parts.

3. Array Reshaping and Conversion

reshape()

Changes array shape.

```
np.reshape([1,2,3,4],(2,2))
```

Output

```
array([[1, 2],  
       [3, 4]])
```

Explanation

Converts a one-dimensional array into a 2×2 matrix.

1D to 2D Conversion

```
np.array([1,2,3,4]).reshape(2,2)
```

Output

```
array([[1, 2],  
       [3, 4]])
```

Explanation

Reshapes a 1D array into a 2D array.

transpose()

Interchanges rows and columns.

```
np.transpose([[1,2],[3,4]])
```

Output

```
array([[1, 3],  
       [2, 4]])
```

Explanation

Rows become columns and columns become rows.

4. Array Operations

append()

Adds elements into arrays.

```
np.append([1,2,3],4)
```

Output

```
array([1, 2, 3, 4])
```

Explanation

Adds value 4 at the end of the array.

`delete()`

Deletes array elements.

`np.delete([1,2,3,4],1)`

Output

`array([1, 3, 4])`

Explanation

Deletes the element at index position 1.

`concatenate()`

Combines arrays together.

`np.concatenate(([1,2],[3,4]))`

Output

`array([1, 2, 3, 4])`

Explanation

Joins two arrays into a single array.

`vstack()`

Stacks arrays vertically.

`np.vstack(([1,2],[3,4]))`

Output

`array([[1, 2],
 [3, 4]])`

Explanation

Places arrays one below another.

`hstack()`

Stacks arrays horizontally.

```
np.hstack(([1,2],[3,4]))
```

Output

```
array([1, 2, 3, 4])
```

Explanation

Places arrays side by side.

```
split()
```

Splits arrays into equal parts.

```
np.split(np.array([1,2,3,4]),2)
```

Output

```
[array([1, 2]), array([3, 4])]
```

Explanation

Splits the array into 2 equal arrays.

5. Mathematical Functions

```
sum()
```

```
np.sum([1,2,3])
```

Output

```
6
```

Explanation

Adds all elements in the array.

```
mean()
```

```
np.mean([1,2,3])
```

Output

```
2.0
```

Explanation

Finds the average value.

`median()`

`np.median([1,2,3])`

Output

2.0

Explanation

Finds the middle value of the array.

`max()`

`np.max([1,2,3])`

Output

3

Explanation

Finds the maximum value.

`min()`

`np.min([1,2,3])`

Output

1

Explanation

Finds the minimum value.

`sqrt()`

`np.sqrt([1,4,9])`

Output

`array([1., 2., 3.])`

Explanation

Finds square root values.

6. Random Functions

Random Values

```
np.random.rand(2,2)
```

Output

```
array([[0.45, 0.78],  
       [0.12, 0.91]])
```

Explanation

Generates random decimal values between 0 and 1. Output changes each time.

Random Integer Values

```
np.random.randint(1,10,(2,2))
```

Output

```
array([[4, 7],  
       [2, 9]])
```

Explanation

Generates random integers between 1 and 9. Output changes each time.

7. Array Information Functions

shape

```
np.array([[1,2],[3,4]]).shape
```

Output

```
(2, 2)
```

Explanation

Shows rows and columns of the array.

size

```
np.size([1,2,3,4])
```

Output

4

Explanation

Finds total number of elements in the array.

dtype

```
np.array([1,2,3]).dtype
```

Output

```
dtype('int64')
```

Explanation

Shows datatype of the array elements.

ndim

```
np.array([[1,2],[3,4]]).ndim
```

Output

2

Explanation

Shows number of dimensions of the array.

8. 2D Array Operations

```
import numpy as np
```

```
arr1=np.array([1,2,3,4,5,6,7])
```

```
arr2=np.array([8,9,1,2,3,5,7])
```

```
final=np.array([arr1,arr2])
```

```
print("2d array:",final)
```

```
print(np.sum(final,axis=1))
```

```
print(np.mean(final, axis=0))
```

```
print(final.ndim)
```

Output

```
2d array: [[1 2 3 4 5 6 7]
           [8 9 1 2 3 5 7]]
```

```
[28 35]
```

```
[4.5 5.5 2.  3.  4.  5.5 7. ]
```

2

Explanation

- Converts two arrays into one 2D array
 - Performs row-wise sum using axis=1
 - Performs column-wise mean using axis=0
 - Finds dimension using ndim
-

9. Random Array, Reshape, Vertical Stack and Horizontal Stack

```
import numpy as np
```

```
arr = np.random.randint(1,10,9)
```

```
print("original array:",arr)
```

```
v=np.vstack(arr)
```

```
print("Vertical:",v)
```

```
h=np.hstack(arr)
```

```
print("Horizontal:",h)
```

```
final = np.reshape(arr,(3,3))
```

```
print(final)
```

Explanation

- Generates random integer values
 - Vertically stacks arrays
 - Horizontally stacks arrays
 - Converts 1D array into 3×3 matrix using reshape
-

10. Matrix Operations using Ones Matrix and Identity Matrix

```
import numpy as np

one=np.ones((3,3))

eye=np.eye(3)

print("Ones matrix",one)

print("Eye matrix:",eye)

s=np.sum([one,eye])

m=np.matmul(one,eye)

print(s,m)
```

Output

```
Ones matrix [[1. 1. 1.]
             [1. 1. 1.]
             [1. 1. 1.]]
```

```
Eye matrix: [[1. 0. 0.]
             [0. 1. 0.]
             [0. 0. 1.]]
```

12.0

```
[[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]
```

Explanation

- Creates ones matrix and identity matrix

- Finds total sum of matrices using `sum()`
 - Performs matrix multiplication using `matmul()`
 - Multiplication with identity matrix keeps the original matrix unchanged
-

Conclusion

Learned various NumPy functions including array creation, reshaping, mathematical operations, matrix multiplication, stacking, splitting, random value generation, and 2D array manipulations using Google Colab.